Day 4 Task

```
import java.util.*;
class InvalidInputException extends Exception {
    public InvalidInputException(String message) {
        super(message);
    }
}

class Student {
    public final int id;
    public final String name;
     final double marks;

    public Student(int id, String name, double marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public double getMarks() { return marks; }

    @Override
    public String toString() {
        return String.format("ID: %-5d | Name: %-20s | Marks: %.2f", id, name, marks);
```

```
    }
}

public class StudentRecordManager {

    // ArrayList keeps records in order (good for sorting and listing)
    public final List<Student> students = new ArrayList<>();
    // HashMap gives fast lookup by ID
    public final Map<Integer, Student> idIndex = new HashMap<>();

    public void addStudent(int id, String name, double marks) throws InvalidInputException {
        if (id <= 0) {
            throw new InvalidInputException("ID must be a positive number.");
        }
        if (idIndex.containsKey(id)) {
            throw new InvalidInputException("A record with ID " + id + " already exists.");
        }
        if (name == null || name.trim().isEmpty()) {
            throw new InvalidInputException("Name cannot be empty.");
        }
        if (!name.trim().matches("[A-Za-z ]+")) {
            throw new InvalidInputException("Name should contain only letters and spaces.");
        }
        if (marks < 0 || marks > 100) {
            throw new InvalidInputException("Marks must be between 0 and 100.");
        }
        Student s = new Student(id, name.trim(), marks);
        students.add(s);
```

```
        idIndex.put(id, s);
    }

    public Student searchById(int id) throws InvalidInputException {
        Student s = idIndex.get(id);
        if (s == null) {
            throw new InvalidInputException("No record found with ID " + id + ".");
        }
        return s;
    }

    // Partial, case-insensitive match; may return several students
    public List<Student> searchByName(String name) throws InvalidInputException {
        if (name == null || name.trim().isEmpty()) {
            throw new InvalidInputException("Search name cannot be empty.");
        }
        String key = name.trim().toLowerCase();
        List<Student> result = new ArrayList<>();
        for (Student s : students) {
            if (s.getName().toLowerCase().contains(key)) {
                result.add(s);
            }
        }
        if (result.isEmpty()) {
            throw new InvalidInputException("No record found matching \"" + name.trim() +
"\".");
        }
        return result;
```

```
}

public void deleteById(int id) throws InvalidInputException {
    Student s = searchById(id);
    students.remove(s);
    idIndex.remove(id);
}

public void sortByName() {
    students.sort(Comparator.comparing(s -> s.getName().toLowerCase()));
}

// Highest marks first
public void sortByMarks() {
    students.sort(Comparator.comparingDouble(Student::getMarks).reversed());
}

public void displayAll() {
    if (students.isEmpty()) {
        System.out.println("No records to display.");
        return;
    }
    for (Student s : students) {
        System.out.println(s);
    }
}

// ---------- Input helpers: wrap parsing errors in exceptions ----------
```

```
public static int readInt(Scanner sc, String prompt) throws InvalidInputException {
    System.out.print(prompt);
    String line = sc.nextLine().trim();
    try {
        return Integer.parseInt(line);
    } catch (NumberFormatException e) {
        throw new InvalidInputException("\"" + line + "\" is not a valid whole number.");
    }
}

public static double readDouble(Scanner sc, String prompt) throws InvalidInputException {
    System.out.print(prompt);
    String line = sc.nextLine().trim();
    try {
        return Double.parseDouble(line);
    } catch (NumberFormatException e) {
        throw new InvalidInputException("\"" + line + "\" is not a valid number.");
    }
}

public static String readString(Scanner sc, String prompt) {
    System.out.print(prompt);
    return sc.nextLine();
}

public static void printMenu() {
    System.out.println("\n===== Student Record Manager =====");
```

```
    System.out.println("1. Add student");
    System.out.println("2. Display all students");
    System.out.println("3. Search by ID");
    System.out.println("4. Search by name");
    System.out.println("5. Sort by name (A-Z)");
    System.out.println("6. Sort by marks (high to low)");
    System.out.println("7. Delete student");
    System.out.println("0. Exit");
}

public static void main(String[] args) {
    StudentRecordManager manager = new StudentRecordManager();
    Scanner sc = new Scanner(System.in);
    boolean running = true;

    while (running) {
        printMenu();
        try {
            int choice = readInt(sc, "Enter choice: ");
            switch (choice) {
                case 1: {
                    int id = readInt(sc, "Enter ID: ");
                    String name = readString(sc, "Enter name: ");
                    double marks = readDouble(sc, "Enter marks (0-100): ");
                    manager.addStudent(id, name, marks);
                    System.out.println("Record added successfully.");
                    break;
                }
```

```
case 2:
    manager.displayAll();
    break;
case 3: {
    int id = readInt(sc, "Enter ID to search: ");
    System.out.println(manager.searchById(id));
    break;
}
case 4: {
    String name = readString(sc, "Enter name to search: ");
    for (Student s : manager.searchByName(name)) {
        System.out.println(s);
    }
    break;
}
case 5:
    manager.sortByName();
    System.out.println("Sorted by name:");
    manager.displayAll();
    break;
case 6:
    manager.sortByMarks();
    System.out.println("Sorted by marks:");
    manager.displayAll();
    break;
case 7: {
    int id = readInt(sc, "Enter ID to delete: ");
    manager.deleteById(id);
```

```
                System.out.println("Record deleted.");
                break;
            }
            case 0:
                running = false;
                System.out.println("Goodbye!");
                break;
            default:
                throw new InvalidInputException("Please choose a number from 0 to 7.");
          }
        } catch (InvalidInputException e) {
          System.out.println("Error: " + e.getMessage());
        } catch (NoSuchElementException e) {
          // Input stream closed (e.g. Ctrl+D)
          System.out.println("\nInput ended. Exiting.");
          running = false;
        }
      }
      sc.close();
    }
}
```